

Placement Portal Application V2 (MAD-II)

1. Student Details

- **Name:** Sahil Kumar
- **Roll Number:** 24f2000164
- **Email:** 24f2000164@ds.study.iitm.ac.in
- **About Me:** I am a student at IIT Madras Online BS Degree Program with interest in full-stack web development. I enjoy building real-world applications that solve practical problems using modern frameworks and technologies.

2. Project Overview

Project Title: Placement Portal Application V2 (MAD-II)

Problem Statement

Institutes currently rely on manual processes like spreadsheets and emails to manage campus placements, making it difficult to coordinate company approvals, student registrations, placement drives, and application tracking efficiently. The goal is to build a role-based web application that enables Admin, Companies, and Students to interact with a centralized placement system — streamlining drive creation, application management, interview scheduling, and placement reporting.

Approach

The application was built using Flask as the REST API backend and Vue.js as the reactive frontend. A JWT-based authentication system handles role-based access for three roles — Admin, Company, and Student. SQLAlchemy manages all data relationships. Redis is used for API caching and as a Celery broker for asynchronous background jobs including daily reminders, monthly reports, and CSV exports. Chart.js provides analytics visualizations on the admin dashboard.

3. AI/LLM Declaration

AI was utilized as a development assistant for structural and foundational code components. Final integration, business logic, and debugging were performed manually.

Component	AI Tool Used	Purpose	Usage %
SQLAlchemy Models	Chatgpt	Model definitions, relationships, FK setup	~8%

JWT Auth & Decorators	Claude	Token generation, role_required decorator	~6%
Celery Task Setup	Chatgpt	Celery config, beat schedule, task structure	~5%
Vue.js Components	Claude	Component structure, Vuex store, axios setup	~8%
Redis Caching	Claude	Cache key design, expiry strategy	~3%
Debugging & Fixes	Chagpt	Error diagnosis, import fixes, logic bugs	~5%
Documentation	Claude	Inline comments, README formatting	~3%
Total			~38%

4. Technologies and Frameworks Used

Technology	Purpose
Flask	REST API backend framework
Vue.js (CLI)	Reactive frontend SPA
SQLAlchemy	ORM for SQLite database
SQLite	Local relational database
JWT (Flask-JWT-Extended)	Token-based authentication
Redis	API caching + Celery message broker
Celery	Async background job processing
Bootstrap 5	Frontend styling and layout
Chart.js	Analytics and placement charts
Vuex	Frontend state management
Flask-Mail	Email notifications
Axios	HTTP client for API calls

5. Database Schema

The relational database is structured to support complex role-based interactions and placement tracking.

Tables and Attributes

1. **User:** id, email, password_hash, role, is_active, is_blacklisted
2. **Student:** id, user_id (FK), full_name, branch, cgpa, year, skills, resume_path
3. **Company:** id, user_id (FK), company_name, industry, location, approval_status
4. **PlacementDrive:** id, company_id (FK), job_title, min_cgpa, eligible_branches, deadline, status
5. **Application:** id, student_id (FK), drive_id (FK), status, applied_at
6. **Interview:** id, application_id (FK), scheduled_at, mode, venue, result
7. **Notification:** id, user_id (FK), message, type, is_read
8. **AdminLog:** id, admin_id (FK), action, target_type, target_id, timestamp
9. **Report:** id, month, year, total_drives, total_applications, total_selected, report_path

Core Relationships

- **User → Student / Company:** One-to-One
- **Company → PlacementDrive:** One-to-Many
- **Student → Application:** One-to-Many
- **PlacementDrive → Application:** One-to-Many
- **Application → Interview:** One-to-One

ER diagram: [Link](#)

6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST ▾	Login for all roles
/api/auth/register/student	POST ▾	Student registration
/api/auth/register/company	POST ▾	Company registration
/api/admin/dashboard	GET ▾	Admin stats
/api/admin/companies/:id/approve	PUT ▾	Approve company
/api/admin/drives/:id/approve	PUT ▾	Approve drive

/api/admin/students/:id/blacklist	PUT ▾	Blacklist student
/api/company/drives	GET/POST ▾	List or create drives
/api/company/applications/:id/status	PUT ▾	Update application status
/api/company/applications/:id/interview	POST ▾	Schedule interview
/api/student/drives	GET ▾	View approved drives
/api/student/apply/:drive_id	POST ▾	Apply to drive
/api/student/applications	GET ▾	View own applications
/api/student/history	GET ▾	Placement history
/api/student/export/csv	POST ▾	Trigger CSV export
/api/admin/reports/generate	POST ▾	Trigger monthly report
/api/analytics/admin/overview	GET ▾	Charts data
/api/public/stats	GET ▾	Public landing stats

7. Architecture and Core Features

System Architecture

The project follows a decoupled client-server architecture. The **Backend** is organized into models, routes (Flask Blueprints), services, and Celery tasks. The **Frontend** utilizes Vue.js with Vuex for state management and Vue Router with role guards for secure navigation.

Key Features Implemented

- **Role-Based Access Control:** Secure JWT authentication for Admins, Companies, and Students.
- **Admin Governance:** Dashboard for approvals, student blacklisting, and audit logs.
- **Drive Management:** End-to-end workflow from company drive creation to student application.

- **Automated Communication:** Daily interview and deadline reminders via Celery and Flask-Mail.
- **Advanced Reporting:** Monthly HTML reports and asynchronous CSV exports.
- **Performance Optimization:** Redis caching on heavy API endpoints to ensure low latency.

Additional Features:

- **Data Visualization:** Comprehensive analytics via Chart.js focusing on recruitment funnels and skill demands.
- **ATS Features:** Built-in resume checker for students and companies to streamline screening

8. Presentation and Links

Video Presentation

The project demonstration video can be accessed via the following :[Link](#)